

# RC4-Verschlüsselung

Yusuf Enes

HTW-Aalen Elektronik und Informatik

Tamer

Cryptography Engineering

## Abstract

Es geht um die RC4-Verschlüsselung, deren Algorithmus aus einer Transposition besteht und auf eine XOR-Operation basiert.

1.

$MasterKey = keyGeneration()$

2.

$PlainField \oplus MasterKey$

## 1 Ablauf des Algorithmus

In diesen Fall muss bei diesem Algorithmus ein Masterkey generiert werden. Dieser wird aus einer Box bzw. aus einem internen Speicher generiert. Die interne Box besteht aus 256 Bytes. Die Struktur oder bzw. die Anpassung ist eine Matrix, wodabei folgendermaßen gilt:

$$8 \times 32$$

Die Werte in der internen Box werden permutiert. Die Permutation wird mit folgendem Pseudocode beschrieben.

```
for(int i = 0; i<256; i++)
{
    j = (j + this->I_Box[i] +
        secret[i % keyLength] ) % 256;
    this->swap(i, j);
}
```

Nachdem der j index berechnet wurde, werden gemäß den beiden Parameter die Byte-Werte ausgetauscht. Beim Debuggen wurde für das Beispiel:

1. PlainText = Hello new World 1

2. Secret = passphrase

folgendes beobachtet:

## 1.1 Der interne Speicher

Damit befindet sich im internen Speicher mehrere wiederholende Vertauschungs-Operationen. Diese werden wie im Pseudocode als Swapping bezeichnet. Um den internen Spe-

icher rauszulesen bzw. zu Debuggen wurde der SELECT-Algorithmus programmiert:

```
void rc4::select(int s)
{
    int q = 0;
    for(int i = 0; i<8; i++)
    {
        for(int j = 0x00; j<= 0x1f; j++)
        {
            if( this->I_Box[i * 32 + j] == s)
            {
                printf("%s%02x ", KRED, this->I_Box[q]);
                q++;
                continue;
            }
            printf("%s%02x ", KGRN, this->I_Box[q]);
            q++;
        }
        printf("\n");
    }
}
```

## 1.2 Schlüsselgenerierung sowie auch die XOR-Operation

Bei der Schlüsselgenerierung wird ein weiterer Algorithmus verwendet, dieser sieht in C++ folgendermaßen aus:

```
for (int n = 0; n < this->Length; n++)
{
    i = (i + 1) % 256;
    j = (j + this->I_Box[i]) % 256;
    this->swap(i, j);
    this->MasterKey[n] = this->I_Box[(this->I_Box[i] + this->I_Box[j]) % 256];
    uint8_t y = *(this->plainField + n);
    uint8_t q = y ^ this->MasterKey[n];
    memset(this->cipherField+n, q, 1 * sizeof(uint8_t));
}
```

Der interne Zustand wird wieder permutiert, jedoch wird bei der Permutation das Secret bzw. das Passwort nicht erneut eingeflossen, da dieser während der Startphase bereits verwendet worden ist. Das Ergebnis der XOR-Operation und der Transposition werden in der MasterKey-Variable gespeichert. Dieser ist ein Array und kann mit einem Zähler bzw. einem Index zugegriffen werden. Aufgrund eines vorhandenen MasterKeys kann direkt mit einer XOR-Operation die Entschlüsselung erfolgen.

```
void rc4::decrypt()
{
    printf(" DECRYPTION CALLED \n");
    for(int i = 0; i<19; i++)
    {
        this->decryptedField[i] = this->MasterKey[i] ^ *(this->cipherField+i);
    }
    return;
}
```

Damit kann der RC4-Algorithmus die Objektorientierten-Programmierung zu nutze machen, weshalb für die Entschlüsselung nur der XOR-Operator verwendet wird, und die Vertauschungs-Operationen im internen Speicher damit nicht stattfinden müssen. Der RC4-Headerfile kann in C++ folgendermaßen aussehen:

```
class rc4{
public:
    uint8_t I_Box[256];
    uint8_t * plainField;
    uint8_t * keyField;
    uint8_t * cipherField;
    uint8_t * MasterKey;
    uint8_t * decryptedField;
    int keyLength;
    int plainTextLength;
    int Length;
    rc4(const char *a, const char * b);
    void setPlainField( const char * buf);
    void setKeyField(uint8_t * key);
    void readPlainField();
    void readCipherField();
    void readKeyField();
    void readDecryptedField();
    void initialize();
    void select(int s);
    void printI_Box();
    void encrypt(uint8_t K[]);
    void decrypt();
    void swap(int i, int j);
};
```